ENTERPRISE DATA ENGINEERING CAPSTONE PROJECT

ENTERPRISE SUPPLY CHAIN DATA PLATFORM (ESCDP)

Project Report: Sprint 0, Sprint 1, Sprint 2 and Sprint 3

Submitted by

ASWIN C

DIYA SUNIL

MAGIN K S

Enterprise Data Engineering Capstone

Project coverage: Initiation, ingestion, data quality, warehouse design, simulated ingestion, governance preparation and analytics

Document Information

Item	Details
Project	Enterprise Supply Chain Data Platform (ESCDP)
Use case	UC18: Building a Unified Supply Chain & Logistics Analytics Platform
Organization in case study	ABC Global Manufacturing Ltd.
Sprints covered	Sprint 0, Sprint 1, Sprint 2 and Sprint 3
Prepared by	ASWIN C; DIYA SUNIL; MAGIN K S
Primary tools	Pentaho Data Integration, PostgreSQL, Python/Pandas, Git/GitHub and Power BI
Report basis	UC18 project brief, supplied project presentation and supplied master-job and star-schema screenshots

Contents

- 1. Executive Summary
- 2. Introduction and Business Background
- 3. Problem Statement
- 4. Objectives and Scope
- 5. Technology Stack and Overall Architecture
- 6. Sprint 0 — Project Initiation and Architecture
- 7. Sprint 1 — Data Discovery and Ingestion
- 8. Sprint 2 — Data Profiling, Quality and Data Warehouse
- 9. Sprint 3 — Governance, Orchestration, Simulated Ingestion and Analytics
- 10. Master Job and Star Schema Evidence
- 11. Validation, Testing and Known Limitations
- 12. Deliverables Summary
- 13. Conclusion and Future Enhancements
- Appendix A — Suggested Demonstration Checklist
- Appendix B — Source Materials

1. Executive Summary

The Enterprise Supply Chain Data Platform (ESCDP) project addresses the challenge of fragmented supply-chain information across multiple enterprise systems. The UC18 case study describes ABC Global Manufacturing Ltd., whose operations span manufacturing plants, distribution centres, warehouses, logistics partners and retail outlets. Data from these functions must be integrated and prepared in a consistent form to support reporting and operational decision-making.

The project is organized into four sprints. Sprint 0 establishes the business context, scope, requirements and high-level architecture. Sprint 1 focuses on source discovery, data dictionaries, Pentaho ETL pipelines and PostgreSQL staging. Sprint 2 covers Python-based profiling and cleaning, data-quality reporting, SQL scripts and dimensional modelling. Sprint 3 extends the platform with ETL orchestration, data-lineage and governance deliverables, simulated incremental ingestion, validation and Power BI analytics.

The supplied project materials document a warehouse model with eight dimensions and six fact tables, together with a Pentaho master job that sequences dimension and fact loading. The project presentation also describes a Python replay process that reads processed CSV records and inserts them into staging.realtime_events. This is simulated streaming from existing files, not a live feed from external enterprise systems. Before the solution is treated as production-ready, fact-table grain, source-key mappings, reconciliation totals, governance artifacts and the final Git release should be verified.

2. Introduction and Business Background

ABC Global Manufacturing Ltd. operates a supply chain involving multiple sites, distribution centres, warehouses, logistics partners and retail outlets. Each business function can generate data in its own system, including Supply Chain Management (SCM), Warehouse Management System (WMS), Transportation Management System (TMS), Inventory Management, Enterprise Resource Planning (ERP), Procurement, Vendor Management, Logistics Tracking and Order Fulfilment.

When these systems are managed independently, teams may have difficulty obtaining a unified view of stock, shipments, supplier performance, warehouse utilization and order fulfilment. The ESCDP project proposes a common data foundation: ingest source data, preserve raw records in staging, profile and clean the data, transform it into a dimensional warehouse, and expose appropriate measures for analytics.

3. Problem Statement

Supply-chain data is distributed across heterogeneous source systems and may contain missing values, duplicates, inconsistent formats and incomplete logistics details. Without a standardized ingestion, quality and warehouse process, it is difficult to compare information across business functions or produce reliable end-to-end reports. The project therefore aims to build a unified platform that integrates source datasets, improves data quality, structures analytical data in PostgreSQL and supports traceable reporting.

4. Objectives and Scope

4.1 Objectives

- Integrate supply-chain data from heterogeneous enterprise source domains.
- Create a raw-data landing and PostgreSQL staging layer.
- Profile, clean, validate and standardize logistics, inventory and related datasets using Python/Pandas.
- Design a PostgreSQL dimensional warehouse with fact and dimension tables.

- Develop Pentaho transformations and a master job to coordinate ETL activities.
- Prepare metadata, source-to-target mappings, data lineage and audit/validation documentation as required by UC18.
- Demonstrate incremental event ingestion from processed CSV files into PostgreSQL.
- Enable supply-chain overview, inventory and logistics/delivery analytics in Power BI.
- Maintain project code and artifacts through Git/GitHub and document sprint outcomes.

4.2 Scope

The documented scope includes ten supply-chain source domains, ETL ingestion, staging, Python-based data cleaning, warehouse schema design, orchestration evidence, simulated ingestion and dashboard preparation. The scope does not establish that a live message broker, live ERP/WMS event feed, fully automated production deployment or fully reconciled warehouse facts have been implemented.

5. Technology Stack and Overall Architecture

Technology	Role in the project
Pentaho Data Integration (Spoon)	Visual ETL transformations, data movement, table outputs and job orchestration.
PostgreSQL	Relational staging and analytical warehouse storage.
Python / Pandas	Dataset inspection, profiling, cleaning, standardization and simulated event replay.
Git and GitHub	Version control, collaboration and project artifact history.
Power BI	Supply-chain overview, inventory and logistics/delivery reporting.
Markdown / MS Word	Technical documentation and sprint report preparation.

5.1 High-level data flow

1. Source systems and files: SCM, ERP, WMS, TMS, inventory, procurement, vendor, logistics, order fulfilment and distribution-centre datasets.
2. Ingestion: Pentaho transformations read source files/data and load source-aligned records into PostgreSQL staging structures.
3. Data preparation: Python/Pandas profiling and cleaning workflows produce processed CSV outputs and cleaning reports.
4. Warehouse: PostgreSQL fact and dimension tables provide a structured model for business analysis.
5. Operational demonstration: processed CSVs are replayed by Python into staging.realtime_events, where SQL can inspect counts and recent events.
6. Analytics: Power BI pages present supply-chain overview, inventory and logistics/delivery measures, subject to relationship and KPI validation.

Figure note: the supplied presentation contains a high-level architecture and dashboard references. The two screenshots supplied separately by the project team are included in Section 10 as the requested implementation evidence.

6. Sprint 0 — Project Initiation and Architecture

6.1 Business objective

Sprint 0 prepares the project before implementation. Its purpose is to understand supply-chain processes, identify stakeholders and data sources, define the scope, agree on the architecture and establish project-planning and version-control foundations.

6.2 Activities required by UC18

- Study the supply-chain business processes and the reporting problems caused by fragmented systems.
- Identify business stakeholders and enterprise data sources.
- Prepare the Business Requirements Document (BRD).
- Define the project scope, objectives and expected outcomes.
- Design the high-level solution architecture.
- Create the Git repository and an organized enterprise repository structure.
- Prepare the product backlog and sprint backlog.
- Prepare the project charter and initiation documentation.

6.3 Requirements and expected outputs

Artifact	Purpose / expected contents
Business Requirements Document (BRD)	Business context, stakeholders, requirements, reporting needs, assumptions and acceptance criteria.
Solution architecture	Source systems, Pentaho ingestion, PostgreSQL staging, Python quality processing, warehouse and reporting layers.
Project charter	Purpose, scope, roles, constraints, milestones and high-level risks.
Git repository and structure	Version-controlled ETL jobs, SQL scripts, Python scripts, documentation, and data/configuration guidance.
Product/sprint backlog	Prioritized tasks and sprint-level deliverables.

6.4 Sprint 0 outcome

The supplied presentation establishes the business background, project objectives, technology stack and high-level architecture. UC18 also lists the BRD, solution architecture, Git repository, sprint backlog and project charter as Sprint 0 deliverables. The available materials do not independently show the full contents or repository URL for each planning artifact; these should be attached or referenced in the final submission if they exist as separate files.

7. Sprint 1 — Data Discovery and Ingestion

7.1 Business objective

Sprint 1 builds the raw-data landing zone by discovering the available source systems, documenting fields and ingesting heterogeneous supply-chain data into PostgreSQL staging tables.

7.2 Source domains

No.	Source domain	Example business information described in the project
1	Distribution Center	Warehouse locations and operating

No.	Source domain	Example business information described in the project
		measures.
2	Inventory Management	Order-item quantities, prices, totals and discounts.
3	Logistics Tracking	Delivery status and late-delivery risk.
4	Order Fulfilment	Order dates, status, customer and destination.
5	Procurement	Purchase orders, quantities, prices, freight and dates.
6	Supply Chain Management (SCM)	Market, region and geographic fields.
7	Transportation Management (TMS)	Shipping modes and shipping-duration fields.
8	Vendor Management	Vendor, manufacturing site, terms and delivery dates.
9	Warehouse Management System (WMS)	Product, category, department and order-item details.
10	Enterprise Resource Planning (ERP)	Customer, order, sales and profit fields.

7.3 Implementation activities

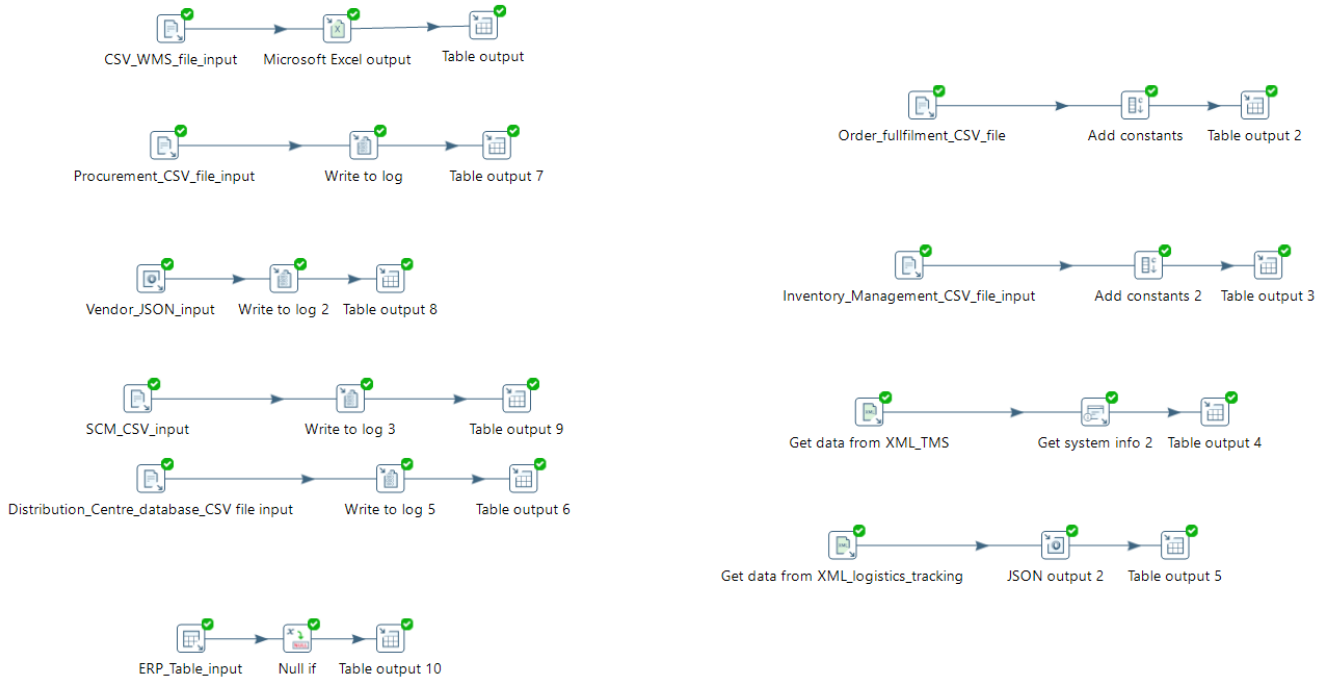
- Analyze each source and prepare a source inventory.
- Create a data dictionary documenting source fields, meanings and formats.
- Develop Pentaho transformations for CSV, Excel, JSON, XML and SQL-derived data as required by UC18.
- Load source-aligned data into PostgreSQL staging tables.
- Configure basic logging and exception handling where implemented.
- Commit successful transformations and supporting artifacts to Git.

7.4 Staging design considerations

The project presentation describes a dedicated staging layer that separates source-level data ingestion from analytical warehouse structures. Source column names may include spaces and punctuation; PostgreSQL requires quoted identifiers for names such as "Order Item Quantity". Landing fields may initially use TEXT where appropriate, followed by validation and type conversion during data preparation. The staging layer should preserve enough source context to support troubleshooting and later source-to-target mapping.

7.5 Sprint 1 deliverables and status

UC18 deliverable	Evidence / interpretation from supplied materials
Source inventory	Ten source domains are identified in the presentation.
Data dictionary	Required by UC18; ensure a field-level version is retained with the project files.
Pentaho ETL pipelines	The supplied master-job screenshot shows sequential dimension and fact load steps; the presentation also references pipeline evidence.
Staging database	PostgreSQL staging structures are described in the presentation.
Git commit history	Required by UC18; include the repository URL or commit evidence in the final submission.



8. Sprint 2 — Data Profiling, Quality and Data Warehouse

8.1 Business objective

Sprint 2 converts raw source data into more trusted, analytics-ready datasets. It combines Python-based profiling and cleaning with PostgreSQL storage design, dimensional modelling, SQL reporting and the preparation of warehouse-loading transformations.

8.2 Profiling and quality checks

- Inspect row and column counts and missing-value patterns for each dataset.
- Identify potential duplicate inventory records and exact duplicate rows.
- Review missing shipment details and inconsistent supplier or warehouse information.
- Normalize common representations of blanks and nulls.
- Remove columns that are entirely missing where detected.
- Trim text whitespace and remove exact duplicate rows.
- Only impute categorical values when a suitable existing category is justified.
- Retain missing dates and identifiers for review instead of inventing values.
- Keep raw files unchanged and write processed outputs under data/processed/.

8.3 Data-quality reports

Report artifact	Purpose
cleaning_summary.csv	Processing status, input/output row counts and dropped-column counts.
cleaning_issues.csv	Documents removed empty columns, duplicate rows and missing values retained for review.
imputation_report.csv	Records the columns filled, number of values and method used.
cleaning_errors.csv	Records source files that failed to process and the associated error.

These reports should be reviewed before loading processed data into the warehouse. Their presence records the cleaning workflow, but does not by itself prove that every issue has been resolved or that all datasets are suitable for production analytics.

8.4 Star-schema design

The supplied SQL and presentation define eight dimensions and six fact tables. Dimensions describe the business context, while facts store measurable business activity. The intended design supports analysis across orders, customers, products, geography, warehouses, vendors, dates and shipping methods.

Type	Table	Main responsibility described in the project
Dimension	dim_date	Calendar attributes such as date, day, month, quarter and year.
Dimension	dim_customer	Customer details and associated geographic attributes.
Dimension	dim_product	Product, category, department and product attributes.
Dimension	dim_geography	City, state, country, region and market.
Dimension	dim_warehouse	Warehouse identity, location, capacity and operating attributes.
Dimension	dim_vendor	Vendor, manufacturing site, terms and delivery-related attributes.
Dimension	dim_shipping_mode	Shipping method/mode attributes.
Dimension	dim_order	Order type, status and date-related attributes.
Fact	fact_order_sales	Sales, profit, benefit and profit-ratio measures.
Fact	fact_inventory	Quantity, prices, totals and discounts.
Fact	fact_shipping	Delivery risk/status and shipping-day measures.
Fact	fact_warehouse	Inventory, capacity, utilization and shipment counts.
Fact	fact_procurement	Purchase quantities, values, prices, freight and insurance.
Fact	fact_delivery	Scheduled/delivered dates, duration and delivery status.

8.5 Fact grain and relationship validation

The project presentation identifies intended grains that must be checked against source identifiers before fact loading. The intended grain is one row per order for fact_order_sales; one row per source inventory/order-item line for fact_inventory; one consolidated shipping record per order for fact_shipping where supported by the source; one warehouse snapshot per date and product for fact_warehouse where available; one procurement line per source ID for fact_procurement; and one procurement/vendor delivery record for fact_delivery.

Foreign-key mappings, duplicate keys, row multiplicity and join cardinality must be checked before aggregating measures. In particular, product-to-inventory and procurement-to-vendor mappings are called out in the supplied presentation as requiring validation. Unverified joins can multiply rows and inflate sales, freight, quantity or other totals.

8.6 Sprint 2 deliverables and status

UC18 deliverable	Evidence / validation note
Data profiling report	Python profiling is part of the documented workflow; retain the generated reports as evidence.
Data quality report	Cleaning summary, issues, imputation and error report filenames are documented.
Star schema	Eight dimensions and six facts are represented in the supplied schema screenshot and presentation.
PostgreSQL data warehouse	DDL/schema design is documented; successful population and reconciliation should be demonstrated separately.
SQL scripts	Warehouse DDL and reporting/verification SQL are expected artifacts.
Git repository updates	Incremental version-control updates are required by UC18.

9. Sprint 3 — Governance, Orchestration, Simulated Ingestion and Analytics

9.1 Business objective

UC18 defines Sprint 3 around data governance, lineage and analytics. The objective is to prepare the platform for enterprise deployment by documenting how data moves from sources to targets, coordinating ETL execution, validating business data and presenting relevant measures through dashboards.

9.2 ETL master job and orchestration

The supplied Pentaho master-job screenshot shows a control flow that starts with loading dimensions and continues through fact-loading steps. The visible sequence includes date, geography, customer, product, shipping-mode, warehouse, vendor, order and shipping-related dimensions/facts, followed by warehouse, procurement and delivery fact steps, a completion/logging step and a success step. The screenshot includes green hops/check indicators and some locked/disabled-looking hops; the exact run configuration should be confirmed in Spoon before interpreting every hop as executed successfully.

The master job is important because it coordinates dependencies: dimension tables generally need to be available before fact tables that reference their surrogate keys are loaded. The final demonstration should show the job configuration, execution result, row counts and any error logs, not only the canvas view.

9.3 Simulated incremental ingestion

The project presentation describes a separate Sprint 3 demonstration path. A Python script named realtime_ingestion.py reads already-processed CSV files from data/processed/, validates and paces the records, and inserts event records into PostgreSQL table staging.realtime_events. The event table includes event_id, source_dataset, source_file, source_row_number, ingestion_time and a JSONB payload.

- Empty or all-blank row checks.
- Duplicate prevention based on source_file and source_row_number.
- Counts for inserted, skipped, invalid and failed records.
- SQL checks for total events, events by source dataset and latest ingestion timestamps.
- Repeated replay loops skip previously ingested source rows where the uniqueness constraint and conflict handling are configured.

This demonstration simulates streaming by replaying existing files at a configurable pace. It is not evidence of a live external event stream, Kafka integration or direct real-time connectivity to ERP/WMS systems.

9.4 Data governance, metadata and lineage

UC18 requires source-to-target mappings, a data-lineage document, a business glossary and a metadata repository. These artifacts should identify each source field, transformation rule, target table/column, key mapping, quality rule and responsible process. They provide traceability from a dashboard measure back to its warehouse fact and source data.

Governance artifact	Recommended contents for this project
Source-to-target mapping	Source file/table and column; target staging/warehouse column; data type; transformation; key mapping; null/quality handling.
Data lineage document	Flow from source domain to Pentaho transformation, staging table, Python cleaning output, warehouse dimension/fact and dashboard measure.
Business glossary	Consistent definitions of terms such as order, sales, profit, inventory unit, late delivery, warehouse utilization, freight and procurement value.
Metadata repository	Dataset/table descriptions, field definitions, owners, refresh process, source, data type and quality notes.
Audit/validation log	Execution timestamp, input/output counts, rejected records, errors, reconciliation results and job status.

The supplied materials define these as Sprint 3 requirements, but do not include a complete lineage document or glossary in the files provided for this report. They should be attached if already prepared, or completed before claiming full governance acceptance.

9.5 Validation and reconciliation

The presentation describes SQL checks on event counts, latest events, source dataset and timestamps. It also records one checkpoint with 680 records in staging.realtime_events and zero logged validation errors. These are checkpoint observations from a particular run, not guaranteed totals for every execution. They should be accompanied by query output or logs if included in a final demonstration.

Warehouse validation should reconcile source and target row counts, check primary and foreign keys, test duplicate handling, verify date conversion and measure calculations, and compare Power BI totals with approved SQL queries. A zero-error event-table checkpoint does not prove that all warehouse facts and dashboard relationships are correct.

9.6 Power BI analytics

The project presentation references dashboard pages for Supply Chain Overview, Inventory Analytics, and Logistics and Delivery. Example measures include total orders and sales, procurement value, inventory units and capacity, delivery status, shipping modes, freight and insurance costs. The KPI definitions, filter behavior, relationships and totals must be validated against source and warehouse data before presenting them as final business results.

9.7 Sprint 3 deliverables and status

UC18 deliverable	Status interpretation based on supplied materials
Data lineage document	Required by UC18; a complete document was not included among the supplied files.
Metadata repository	Required by UC18; confirm that a maintained artifact exists.

UC18 deliverable	Status interpretation based on supplied materials
Pentaho jobs	A master-job screenshot was supplied; execution logs and successful row-count evidence should accompany it.
Validation report	Event-ingestion checks are described; full warehouse reconciliation remains a validation priority.
Power BI dashboard	Dashboard pages are referenced in the presentation; KPI definitions and relationships require verification.
Final Git release	Required by UC18; provide repository URL, release tag or commit reference if available.
Project report and presentation	This report consolidates the supplied project requirements and evidence; presentation was supplied.

10. Master Job and Star Schema Evidence

10.1 Pentaho master job

Figure 1 presents the supplied Pentaho master job. It visually documents the intended orchestration from dimension loading through fact loading and final completion/success steps. It is evidence of the job design; a successful execution should additionally be supported by Spoon logs and target-table row counts.

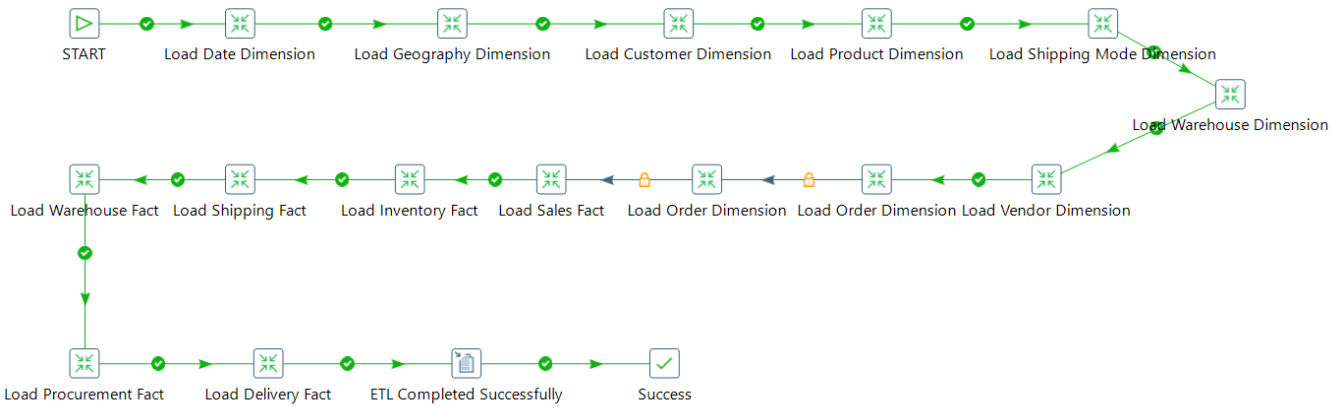


Figure 1. Pentaho master job for the ESCDP warehouse loading workflow (screenshot supplied by the project team).

10.2 PostgreSQL star schema

Figure 2 presents the supplied PostgreSQL schema diagram. It shows the dimension tables and fact tables used to structure analytical data. The model should be reviewed alongside the SQL DDL to verify primary keys, foreign keys, nullable mappings, grain and join paths.

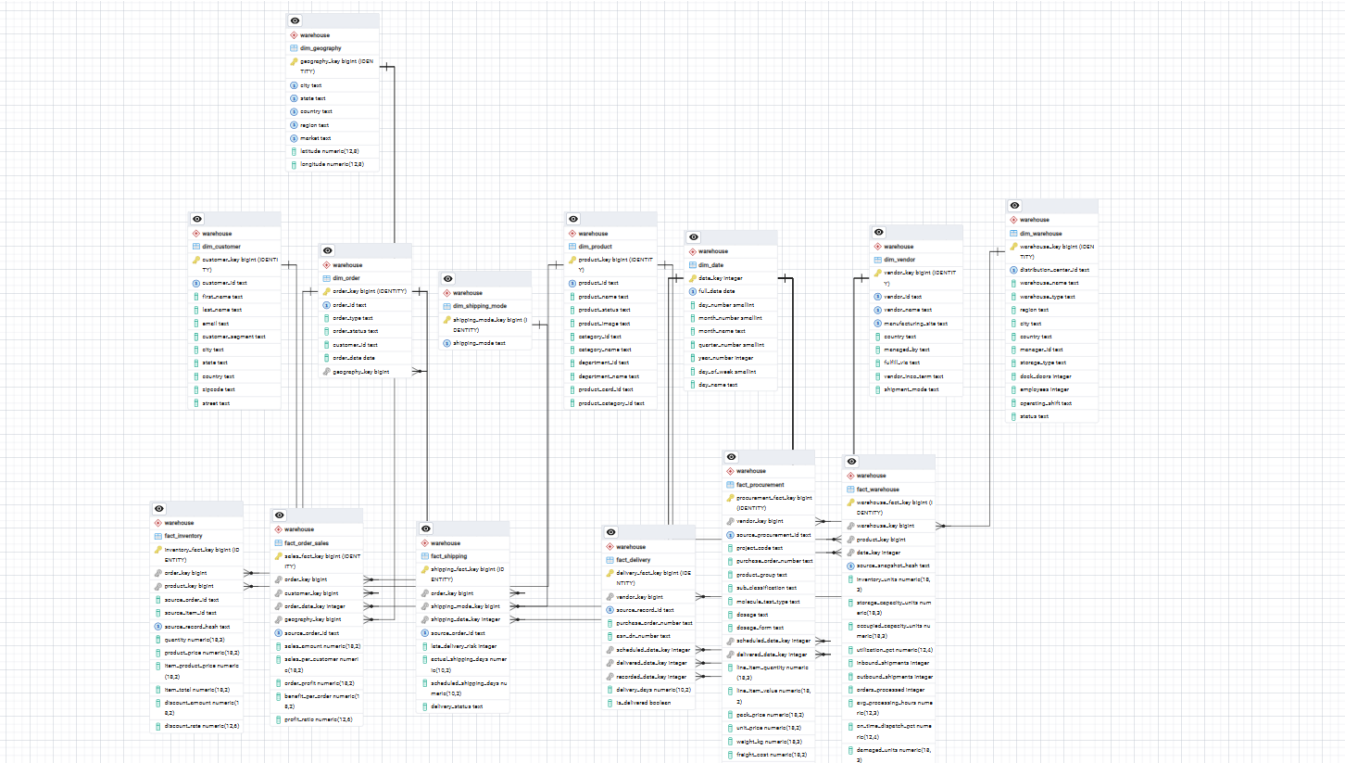


Figure 2. PostgreSQL star-schema diagram showing the ESCDP dimension and fact tables (screenshot supplied by the project team).

10.3 Reading the model

- Dimension tables hold descriptive attributes used to filter, group and interpret measures.
- Fact tables store measurable business activity such as sales, inventory, shipping, warehouse operations, procurement and delivery.
- Surrogate keys and validated foreign-key mappings connect fact records to the correct dimension records.
- Fact grain must be consistent with source-level business events to prevent double counting.
- The existence of the schema diagram confirms the design representation, but successful fact loading and accurate reporting require execution and reconciliation evidence.

11. Validation, Testing and Known Limitations

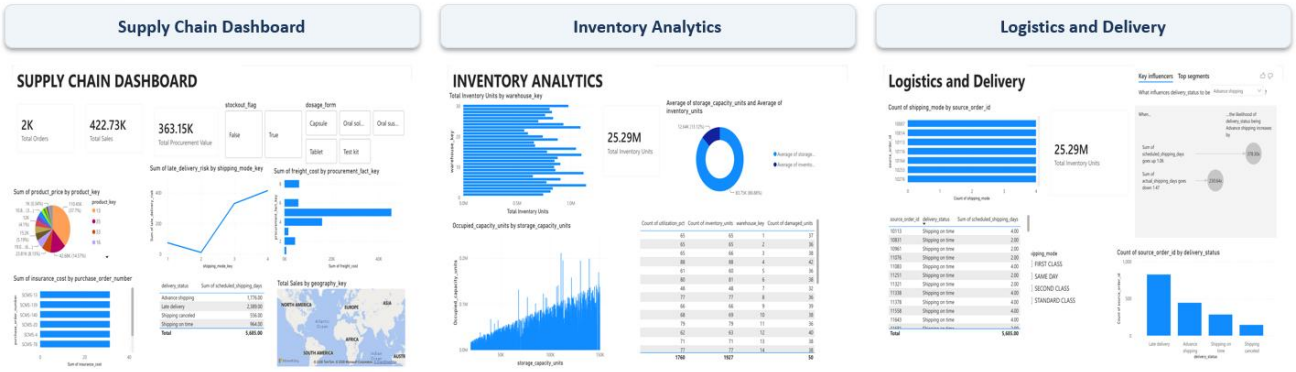
11.1 Recommended validation checklist

- Confirm every expected source file is discovered and its ingestion status is logged.
- Compare source row counts with staging and processed-output row counts.
- Check missing values, duplicates, invalid dates, invalid numeric values and unexpected categories.
- Confirm that cleaning does not silently invent identifiers or dates.
- Validate dimension business keys and surrogate-key uniqueness.
- Validate each fact table's grain and foreign-key mappings.
- Compare warehouse totals with source-level aggregates using documented rules.
- Run the Pentaho master job and retain execution logs, errors and target row counts.
- Replay the processed CSVs more than once and confirm duplicate prevention behaves as intended.
- Verify Power BI relationships, filter directions, KPI definitions and SQL-to-dashboard totals.
- Record unresolved issues and the responsible action before final release.

11.2 Limitations stated or implied by the available project evidence

- The real-time demonstration is simulated by replaying processed CSV files; it is not a live external feed.
- The presentation says fact-to-dimension joins and source-key mappings must be validated.
- The intended grain of each fact table must be checked against actual source identifiers.
- The reported 680-event checkpoint and zero logged validation errors apply to one observed run only.
- The complete source-to-target mapping, lineage document, business glossary, metadata repository, final Git release and full reconciliation evidence were not present as standalone artifacts in the supplied files.
- Dashboard values should not be treated as final business results until their definitions and relationships are reconciled.

11.3 POWER BI DASHBOARD



12. Deliverables Summary

Sprint	Core UC18 deliverables	Evidence / follow-up
Sprint 0	BRD, solution architecture, Git repository, sprint backlog, project charter.	Architecture and project framing are described in the presentation; attach full planning artifacts and repository link if available.
Sprint 1	Source inventory, data dictionary, Pentaho pipelines, staging database, Git commit history.	Ten source domains and staging approach are documented; retain field-level dictionary, pipeline files and run logs.
Sprint 2	Profiling report, data-quality report, star schema, PostgreSQL warehouse, SQL scripts, Git updates.	Cleaning report outputs are described and schema diagram supplied; verify actual outputs, table population and reconciliation.
Sprint 3	Lineage document, metadata repository, Pentaho jobs, validation report, Power BI dashboard, final Git release, project report and presentation.	Master-job and schema screenshots supplied; simulated ingestion and dashboard pages described; complete governance and final-release evidence as needed.

13. Conclusion and Future Enhancements

The ESCDP project defines an end-to-end approach for consolidating supply-chain data from ten enterprise source domains. The work described across Sprints 0–3 establishes the project context and architecture,

develops the ingestion and staging approach, introduces data profiling and cleaning, defines a dimensional warehouse and demonstrates orchestration, simulated incremental ingestion and analytics.

The supplied master job and star-schema diagram provide visual evidence of the planned ETL sequence and warehouse structure. The next priority is to verify implementation quality with repeatable execution logs, source-to-target reconciliation, fact-grain checks, validated key mappings and dashboard-to-SQL comparisons. Sprint 3 governance deliverables—particularly lineage, metadata, glossary, validation evidence and final Git release—should be attached or completed to align the submission with UC18.

Future enhancements

- Complete and maintain field-level source-to-target mapping and lineage documentation.
- Automate data-quality checks and enforce documented acceptance thresholds.
- Add repeatable reconciliation queries and store their results with each ETL run.
- Improve observability with job-level audit records, duration, row counts and categorized errors.
- Replace or complement file replay with a message broker/live event integration only if the project scope and infrastructure support it.
- Finalize Power BI KPI definitions and test all relationships against reconciled warehouse outputs.
- Tag a final Git release and archive the approved code, SQL, reports, screenshots and run instructions.

Appendix A — Suggested Demonstration Checklist

7. Show the source inventory and data dictionary.
8. Open a representative Pentaho transformation and explain input, transformation and output steps.
9. Run the master job and show success/failure logs and row counts.
10. Show PostgreSQL staging tables and a sample of loaded records.
11. Run the Python profiling/cleaning workflow and open its summary, issue, imputation and error reports.
12. Show the star-schema diagram and the corresponding SQL DDL.
13. Demonstrate dimension/fact key validation and fact-grain checks.
14. Run realtime_ingestion.py and inspect staging.realtime_events using SQL.
15. Replay the ingestion script and demonstrate duplicate prevention.
16. Open the Power BI overview, inventory and logistics/delivery pages and reconcile selected KPIs with SQL.
17. Show lineage, metadata, glossary, validation report and Git release evidence where available.

Appendix B — Source Materials

- UC18(3).md — Enterprise Data Engineering Capstone Project: UC18 Enterprise Supply Chain Data Platform, sprint roadmap, assigned tasks and deliverables.
- ESCDP_Professional_Sprints0_to3_23_Slides (1).pptx — Project presentation covering business background, architecture, data cleaning, schema, simulated ingestion, validation and analytics.
- Pentaho master-job screenshot supplied with the request.
- PostgreSQL star-schema screenshot supplied with the request.

Note: This report is based on the supplied materials. Where the source documents describe a required deliverable but do not provide the standalone artifact or execution proof, the report identifies it as a follow-up or validation item rather than asserting completion.